

Assignment 15

Program -

```
1]
function NQueens(order)
{
  if (order < 4) {
    console.log('N Queens problem apply for order bigger than 3 ! ');
    return;
  }
  var nQueens = [];
  rowLoop:
  for (var row=0; row<order; row++) {
    // If appear row Less than 0, There is no solution
    if(row < 0){
      console.log('This N Queens problem has no solution ! ');
      break;
    }
    // The first 1 A new one was detected this time 1 line
    if (nQueens[row] === undefined) {
      nQueens[row] = [];
    }
    // A block of programs that run while backtracking
    for (var col=0; col<order; col++) {
      //0 For the position that has been tested and can place the queen
      if (nQueens[row][col] === 0) {
        continue;
      }
      // In the process of backtracking, the position where the queen
      can be put is encountered, which means that the verification of this
      position in the future has not passed and needs to be processed again
      else if (nQueens[row][col] == 1) {
        // Back in time , on 1 The end of the line , I need to keep
        going back
        if (col === order-1) {
          resetRow(nQueens, order, row);
          row = row - 2;
          continue rowLoop;
        }
        // The backtracking line is not at the end of the line , mark
        0, Continue to
        nQueens[row][col] = 0;
        continue;
      }
      // Place the 1 A queen
      nQueens[row][col] = 1;
      // find 1 The queen can be placed in the position, jump down 1
      Line ( 1 Only on the line 1 A queen).
      if (isQueenValid(nQueens, row, col)) {
        continue rowLoop;
      }
      // every 1 There should be 1 The queen, to the end of the column
      has not found a suitable position, indicating that the queen before the
      placement of a problem, need to backtrack!
      else if (col == order-1) {
```

```

        //0 with 1 All indicate that this location has been detected,
so clear the line as undefined
        resetRow(nQueens, order, row);
        // Reduction of 2 Because the end of the loop has a self
addition, which is essentially going back up 1 line
        row = row - 2;
        // Step back into the outer loop and continue
        continue rowLoop;
    } else {
        // If the line is not reached, continue to detect the
undetected
        nQueens[row][col] = 0;
        continue;
    };
}
}
return nQueens;
}
// Back before , Clear line
function resetRow(nQueens, order, row) {
    for (var col=0; col<order; col++) {
        nQueens[row][col] = undefined;
    }
}
// Check if the position can place the queen
function isQueenValid(nQueens, row, col) {
    // Line detection
    for (var i=0; i<col; i++) {
        if (nQueens[row][i] == 1) {
            return false;
        }
    }
    for (var j=1; j<row+1; j++) {
        // Column tests
        // Upper left 45 The degree of
The upper right 45 The degree of
        if (nQueens[row-j][col]==1 || (nQueens[row-j][col-j]==1) ||
(nQueens[row-j][col+j]==1)) {
            return false;
        }
    }
    return true;
}

function printQ(queens) {
    for (var row=0; row<queens.length; row++) {
        var rowText = '';
        for (var col=0; col<queens.length; col++) {
            if (queens[row][col]==undefined) {
                queens[row][col] = 0;
            }
            rowText = rowText + queens[row][col] + ' ';
        }
        console.log(rowText);
    }
}

var queens = NQueens(8);
printQ(queens);

```

Output -

1	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0